

Fiche de testabilité — requeteCreationJoueur(nom, prenom)

Contexte du projet

Le projet contient :

un serveur de jeu opérationnel,
la librairie fournie

le code client complet

Ce contexte offre un bon niveau de testabilité, car l'ensemble du système est accessible pour concevoir et exécuter les tests.

1. Tableau de synthèse des critères

Critère	Évaluation	Justification synthétique
Observabilité	4/5	La méthode retourne un objet d'identification du joueur, permettant de vérifier facilement la réussite.
Contrôlabilité	4/5	Les paramètres sont entièrement contrôlables. L'état initial du serveur peut être réinitialisé pour les tests.
Disponibilité	4/5	La méthode est invoquable dans tous les contextes, local ou intégré, sans limitation technique.
Stabilité	4/5	Les résultats sont reproductibles à condition de nettoyer la base de données entre tests.

2. Analyse détaillée par critère

Critère	Analyse détaillée	Pistes d'amélioration éventuelles
Observabilité	Le retour contient les données nécessaires pour valider la création. En cas d'erreur côté serveur, les retours ne sont pas toujours explicites.	Ajouter un mécanisme de log côté serveur ou des exceptions plus claires

Contrôlabilité	Les entrées sont contrôlables à 100%. La possibilité de réinitialiser l'état serveur permet de tester différents scénarios (ex : joueur déjà existant).	Automatiser la remise à zéro des données serveur pour tests
Disponibilité	L'appel peut être fait à tout moment du test, sans contraintes particulières.	Aucun
Stabilité	Les tests échouent si la base n'est pas nettoyée, mais deviennent stables avec une bonne isolation des données.	Utiliser une base temporaire ou sandboxée pour les tests

3. Évaluation synthétique

Critère	Note /5	Risques si non maîtrisé
Observabilité	4	Échecs silencieux, difficulté à diagnostiquer erreurs
Contrôlabilité	4	Tests dépendants d'un état serveur non contrôlé
Disponibilité	4	Aucun
Stabilité	4	Tests non reproductibles

Résumé

La méthode est bien testable dans le contexte actuel, sous réserve de disposer d'un environnement serveur réinitialisable et de gérer correctement la base de données de test.

Fiche de testabilité — Constructeur

QuiEstCeClient(hote: String, port: Int)

Contexte du projet

Le projet inclut un serveur de jeu en fonctionnement, la librairie cliente et le code complet du client. Le constructeur établit la connexion avec le serveur via HTTP.

1. Tableau de synthèse des critères

Critère	Évaluation	Justification synthétique
---------	------------	---------------------------

Observabilité	3/5	Pas de retour direct ; seules erreurs d'exception ou logs sont visibles.
Contrôlabilité	4/5	Paramètres contrôlables, mais nécessite un serveur actif au bon hôte/port.
Disponibilité	4/5	Constructeur toujours invoquable si serveur accessible.
Stabilité	4/5	Stabilité liée à la disponibilité réseau et serveur.

2. Analyse détaillée par critère

Critère	Analyse détaillée	Pistes d'amélioration éventuelles
Observabilité	Le constructeur ne retourne rien de spécifique; seul un échec lors de la connexion permet de détecter un problème.	Ajouter un retour d'état ou exceptions plus détaillées
Contrôlabilité	Les paramètres d'hôte et port sont contrôlables, mais le test dépend de la disponibilité du serveur cible.	Mock ou serveur de test local pour plus d'isolation
Disponibilité	Le constructeur peut être appelé dans tous les contextes où le serveur est joignable.	Aucun
Stabilité	Les tests sont reproductibles si le serveur est stable et accessible.	Utiliser un serveur de test dédié

3. Évaluation synthétique

Critère	Note /5	Risques si non maîtrisé
Observabilité	3	Difficulté à diagnostiquer échecs silencieux
Contrôlabilité	4	Dépendance à un serveur disponible
Disponibilité	4	Aucun
Stabilité	4	Tests instables si serveur instable

Fiche de testabilité — `requeteEssai()`

Contexte du projet

Cette méthode teste la communication avec le serveur via une requête simple.

1. Tableau de synthèse des critères

Critère	Évaluation	Justification synthétique
Observabilité	5/5	Retour de chaîne simple vérifiable directement.
Contrôlabilité	5/5	Aucune entrée, test très direct.
Disponibilité	5/5	Toujours disponible si serveur est accessible.
Stabilité	5/5	Résultats toujours identiques à conditions stables serveur.

2. Analyse détaillée par critère

Critère	Analyse détaillée	Pistes d'amélioration éventuelles
Observabilité	Résultat immédiat et facile à vérifier, ce qui facilite le diagnostic.	Aucun
Contrôlabilité	Sans paramètre, méthode simple à tester.	Aucun
Disponibilité	Dépend uniquement de la disponibilité du serveur.	Aucun
Stabilité	Stable si serveur stable.	Aucun

3. Évaluation synthétique

Critère	Note /5	Risques si non maîtrisé
Observabilité	5	Aucun
Contrôlabilité	5	Aucun
Disponibilité	5	Échec si serveur indisponible
Stabilité	5	Résultats non fiables si serveur instable

Contexte du projet

Méthode pour récupérer la liste des identifiants de joueurs existants.

1. Tableau de synthèse des critères

Critère	Évaluation	Justification synthétique
Observabilité	5/5	Retour liste facile à exploiter et à vérifier.
Contrôlabilité	4/5	Pas de paramètres, mais dépend de l'état du serveur (base de joueurs).
Disponibilité	5/5	Toujours disponible si serveur opérationnel.
Stabilité	4/5	Stabilité liée à l'état de la base de joueurs (peut évoluer).

2. Analyse détaillée par critère

Critère	Analyse détaillée	Pistes d'amélioration éventuelles
Observabilité	Le résultat est un ensemble d'identifiants, simple à comparer à une liste attendue.	Ajouter logs détaillés en cas d'erreur serveur
Contrôlabilité	Le test dépend de la population existante sur le serveur (ex : base vide ou remplie).	Prévoir nettoyage de la base entre tests
Disponibilité	Méthode accessible tant que serveur répond.	Aucun
Stabilité	Résultats fluctuants si la base évolue pendant les tests.	Isolation de base ou environnement dédié

3. Évaluation synthétique

Critère	Note /5	Risques si non maîtrisé
Observabilité	5	Difficile de comprendre les échecs sans logs
Contrôlabilité	4	Tests non isolés, dépendance base de données
Disponibilité	5	Aucun

Stabilité	4	Résultats non reproductibles si base modifiée
-----------	---	---

Fiche de testabilité — `requeteJoueur(idJoueur: Int)`

Contexte du projet

Récupère les détails d'un joueur donné via son identifiant.

1. Tableau de synthèse des critères

Critère	Évaluation	Justification synthétique
Observabilité	5/5	Retour structuré (objet joueur) clair et vérifiable.
Contrôlabilité	4/5	Contrôle total sur l'ID, mais dépend de la présence réelle du joueur.
Disponibilité	5/5	Disponible si serveur et joueur existent.
Stabilité	4/5	Stabilité dépend de la base joueurs et état serveur.

2. Analyse détaillée par critère

Critère	Analyse détaillée	Pistes d'amélioration éventuelles
Observabilité	Objet renvoyé permet une inspection complète des données joueur.	Ajouter des logs d'erreur si idJoueur inconnu
Contrôlabilité	Idem, mais impossible d'accéder à un joueur inexistant.	Permettre création / suppression contrôlée en test
Disponibilité	Disponible dans tous les contextes avec joueur valide.	Aucun
Stabilité	Résultats stables si la base joueurs ne change pas.	Nettoyer la base avant tests

3. Évaluation synthétique

Critère	Note /5	Risques si non maîtrisé
Observabilité	5	Difficultés à détecter des erreurs serveur
Contrôlabilité	4	Test dépend de la présence du joueur
Disponibilité	5	Aucun
Stabilité	4	Tests échouent si base non isolée

Fiche de testabilité — `requeteCreationPartie(idJoueur: Int, cleJoueur: String)`

Contexte du projet

Création d'une partie pour un joueur identifié.

1. Tableau de synthèse des critères

Critère	Évaluation	Justification synthétique
	n	
Observabilité	4/5	Retour d'ID partie, visible, mais peu d'info en cas d'erreur serveur.
Contrôlabilité	4/5	Contrôle total sur paramètres mais nécessite joueur existant et clé valide.
Disponibilité	4/5	Dépend que le joueur existe et est authentifié.
Stabilité	4/5	Stable si base joueurs et parties bien gérées.

2. Analyse détaillée par critère

Critère	Analyse détaillée	Pistes d'amélioration éventuelles
Observabilité	Retour simple, mais erreurs côté serveur peu documentées.	Améliorer retours d'erreur ou logs détaillés
Contrôlabilité	Tests nécessitent des clés valides, donc état serveur maîtrisé.	Outils pour générer clés test et remettre base à zéro

Disponibilité	Accessible si conditions préalables respectées.	Aucun
Stabilité	Stable si base de joueurs maintenue propre.	Nettoyage régulier des données

3. Évaluation synthétique

Critère	Note /5	Risques si non maîtrisé
Observabilité	4	Échec silencieux possible
Contrôlabilité	4	Difficulté à simuler clés invalides ou absentes
Disponibilité	4	Tests non possibles sans joueur valide
Stabilité	4	Tests instables si base incohérente

Fiche de testabilité — requeteEtatPartie(idPartie: Int)

Contexte du projet

Récupère l'état actuel d'une partie.

1. Tableau de synthèse des critères

Critère	Évaluation	Justification synthétique
Observabilité	5/5	Retour objet EtatPartie structuré et facilement vérifiable.
Contrôlabilité	4/5	Contrôle sur idPartie mais dépend état réel de la partie.
Disponibilité	5/5	Disponible si la partie existe.
Stabilité	4/5	Stable si la base parties stable.

2. Analyse détaillée par critère

Critère	Analyse détaillée	Pistes d'amélioration éventuelles
Observabilité	Structure claire pour diagnostic précis.	Ajouter logs côté serveur en cas d'erreur

Contrôlabilité	Nécessite parties préexistantes.	Automatiser création / suppression parties
Disponibilité	Accessible dans tous contextes valides.	Aucun
Stabilité	Reproductible si base stable.	Nettoyer base avant tests

3. Évaluation synthétique

Critère	Note /5	Risques si non maîtrisé
Observabilité	5	Difficulté à diagnostiquer les erreurs
Contrôlabilité	4	Tests non isolés si parties fluctuantes
Disponibilité	5	Aucun
Stabilité	4	Résultats non reproductibles sans isolement

Fiche de testabilité — requetePoserQuestion(idPartie: Int, question: String)

Contexte du projet

Permet à un joueur de poser une question dans une partie donnée.

1. Tableau de synthèse des critères

Critère	Évaluation	Justification synthétique
	n	
Observabilité	4/5	Retour booléen ou chaîne simple, diagnostic possible mais limité.
Contrôlabilité	4/5	Contrôle sur question, mais dépend état serveur (validité partie).
Disponibilité	4/5	Disponible si partie active.

Stabilité	4/5	Stable si partie et serveur stables.
-----------	-----	--------------------------------------

2. Analyse détaillée par critère

Critère	Analyse détaillée	Pistes d'amélioration éventuelles
Observabilité	Retour explicite, mais peu d'info sur raison d'un échec éventuel.	Ajouter codes erreurs ou messages détaillés
Contrôlabilité	Peut être testée avec différentes questions valides ou invalides.	Validation syntaxique côté client ou serveur
Disponibilité	Nécessite partie active et existante.	Aucun
Stabilité	Résultats reproductibles si environnement stable.	Nettoyage / réinitialisation de la partie

3. Évaluation synthétique

Critère	Note /5	Risques si non maîtrisé
Observabilité	4	Difficulté à diagnostiquer certains échecs
Contrôlabilité	4	Impossible de tester questions invalides sans bonne gestion
Disponibilité	4	Échec si partie inactive ou inexistante
Stabilité	4	Résultats non fiables sans environnement stable

Fiche de testabilité —

requeteChercherEncore(idPartie: Int)

Contexte du projet

Indique si le joueur peut continuer à chercher dans la partie.

1. Tableau de synthèse des critères

Critère	Évaluation	Justification synthétique
---------	------------	---------------------------

Observabilité	5/5	Retour booléen clair et vérifiable.
Contrôlabilité	4/5	Contrôle sur idPartie, dépend état partie.
Disponibilité	5/5	Disponible si partie existante.
Stabilité	5/5	Résultats stables si état partie constant.

2. Analyse détaillée par critère

Critère	Analyse détaillée	Pistes d'amélioration éventuelles
Observabilité	Retour direct facile à vérifier lors des tests.	Aucun
Contrôlabilité	Tests possibles avec parties dans différents états.	Automatisation création états différents
Disponibilité	Accessible dans tous contextes valides.	Aucun
Stabilité	Très stable si base stable.	Nettoyage base

3. Évaluation synthétique

Critère	Note /5	Risques si non maîtrisé
Observabilité	5	Aucun
Contrôlabilité	4	Difficulté si états non maîtrisés
Disponibilité	5	Aucun
Stabilité	5	Tests fiables dans un environnement contrôlé

Fiche de testabilité — `requeteTrouve(idPartie: Int, personnage: String)`

Contexte du projet

Permet de valider si le joueur a correctement deviné le personnage.

1. Tableau de synthèse des critères

Critère	Évaluation	Justification synthétique
Observabilité	4/5	Retour booléen clair, mais peu d'info en cas d'erreur.
Contrôlabilité	4/5	Contrôle complet sur personnage, mais dépend partie existante.
Disponibilité	4/5	Disponible si partie en cours.
Stabilité	4/5	Stable si environnement serveur stable.

2. Analyse détaillée par critère

Critère	Analyse détaillée	Pistes d'amélioration éventuelles
Observabilité	Résultat simple mais peu d'information en cas de mauvaise saisie.	Ajouter messages d'erreur plus explicites
Contrôlabilité	Peut être testé avec différents noms de personnages valides/invalides.	Validation côté client ou serveur
Disponibilité	Accessible si partie active.	Aucun
Stabilité	Tests reproductibles si serveur et base stables.	Nettoyage de l'état partie avant tests

3. Évaluation synthétique

Critère	Note /5	Risques si non maîtrisé
Observabilité	4	Échec silencieux difficile à diagnostiquer
Contrôlabilité	4	Test limité si pas de base stable
Disponibilité	4	Non testable sans partie existante
Stabilité	4	Tests non fiables si état non isolé

Contexte du projet

Retourne la liste des parties existantes.

1. Tableau de synthèse des critères

Critère	Évaluation	Justification synthétique
Observabilité	5/5	Liste retournée facile à vérifier.
Contrôlabilité	4/5	Pas de paramètres, dépend état serveur.
Disponibilité	5/5	Toujours disponible si serveur opérationnel.
Stabilité	4/5	Résultats liés à la base parties, donc variable.

2. Analyse détaillée par critère

Critère	Analyse détaillée	Pistes d'amélioration éventuelles
Observabilité	Retour simple à comparer aux attentes.	Ajouter logs en cas d'erreurs serveur
Contrôlabilité	Test dépend de la base partie.	Nettoyage base avant test
Disponibilité	Accessible tout le temps.	Aucun
Stabilité	Variables selon évolution base.	Isolement environnement test

3. Évaluation synthétique

Critère	Note /5	Risques si non maîtrisé
Observabilité	5	Diagnostic difficile sans logs
Contrôlabilité	4	Tests non isolés
Disponibilité	5	Aucun
Stabilité	4	Résultats non reproductibles

Fiche de testabilité —

requeteListePartiesCreees(idJoueur: Int, cleJoueur: String)

Contexte du projet
Retourne la liste des parties créées par un joueur authentifié.

1. Tableau de synthèse des critères

Critère	Évaluation	Justification synthétique
Observabilité	5/5	Liste facilement vérifiable.
Contrôlabilité	4/5	Contrôle sur identifiants, mais dépend état serveur/BD.
Disponibilité	4/5	Accessible si joueur valide.
Stabilité	4/5	Stable selon état base et serveur.

2. Analyse détaillée par critère

Critère	Analyse détaillée	Pistes d'amélioration éventuelles
Observabilité	Retour clair, mais erreurs peu détaillées.	Messages d'erreur détaillés
Contrôlabilité	Nécessite base joueur et clés valides.	Environnements de test dédiés
Disponibilité	Accessible dans contexte valide.	Aucun
Stabilité	Stable si base stable.	Nettoyage régulier

3. Évaluation synthétique

Critère	Note /5	Risques si non maîtrisé
Observabilité	5	Diagnostic limité sans logs
Contrôlabilité	4	Difficulté à gérer clés invalides
Disponibilité	4	Tests limités sans joueur valide

Fiche de testabilité —

requeteRejoindrePartie(idPartie: Int, idJoueur: Int, cleJoueur: String)

Contexte du projet

Permet à un joueur de rejoindre une partie existante.

1. Tableau de synthèse des critères

Critère	Évaluation	Justification synthétique
Observabilité	4/5	Retour EtatPartie complet, diagnostic possible
Contrôlabilité	4/5	Paramètres contrôlables, dépend état partie
Disponibilité	4/5	Accessible si partie ouverte
Stabilité	4/5	Stable si base nettoyée

2. Analyse détaillée par critère

Critère	Analyse détaillée	Pistes d'amélioration éventuelles
Observabilité	Résultat complet mais erreurs peu explicites	Ajouter messages d'erreur et logs serveur
Contrôlabilité	Testable avec joueurs valides et invalides	Outils gestion base et état partie
Disponibilité	Nécessite partie en état de rejoindre	Aucun
Stabilité	Résultats fiables si environnement contrôlé	Nettoyage base

3. Évaluation synthétique

Critère	Note /5	Risques si non maîtrisé
---------	---------	-------------------------

Observabilité	4	Diagnostic limité
Contrôlabilité	4	Limites en cas d'états inconnus
Disponibilité	4	Échec si partie fermée
Stabilité	4	Tests non fiables sans nettoyage

Fiche de testabilité —

requeteChoixPersonnage(idPartie: Int, idJoueur: Int, cleJoueur: String, ligne: Int, colonne: Int)

Contexte du projet

Le joueur choisit un personnage dans la grille du jeu (6 colonnes, 4 lignes).

1. Tableau de synthèse des critères

Critère	Évaluation	Justification synthétique
Observabilité	4/5	Retour EtatPartie complet
Contrôlabilité	4/5	Paramètres contrôlables, contraintes de position
Disponibilité	4/5	Accessible dans partie active
Stabilité	4/5	Stable si base et partie nettoyées

2. Analyse détaillée par critère

Critère	Analyse détaillée	Pistes d'amélioration éventuelles
Observabilité	Résultat détaillé mais peu d'erreurs explicites	Ajouter codes erreurs clairs
Contrôlabilité	Contrôle des indices ligne/colonne requis, tests possibles	Validation stricte des indices
Disponibilité	Partie active requise	Aucun

Stabilité	Tests reproductibles avec nettoyage	Scripts de nettoyage base
-----------	-------------------------------------	---------------------------

3. Évaluation synthétique

Critère	Note /5	Risques si non maîtrisé
Observabilité	4	Difficulté à diagnostiquer erreurs
Contrôlabilité	4	Indices hors limites mal gérés
Disponibilité	4	Échec si partie inactive
Stabilité	4	Tests non reproductibles

requeteGrilleJoueur(idPartie: Int, idJoueur: Int)

Contexte du projet

Récupère la grille de personnages du joueur dans une partie donnée (6 colonnes x 4 lignes).

1. Tableau de synthèse des critères

Critère	Évaluation	Justification synthétique
Observabilité	5/5	Retour d'une grille complète, facile à analyser
Contrôlabilité	4/5	Paramètres contrôlables, dépend base et état partie
Disponibilité	5/5	Accessible si partie et joueur valides
Stabilité	5/5	Résultats stables dans environnement contrôlé

2. Analyse détaillée par critère

Critère	Analyse détaillée	Pistes d'amélioration éventuelles
Observabilité	Grille complète structurée en listes, permet vérification facile	Ajouter logs en cas d'erreur ou grille vide

Contrôlabilité	Contrôle complet sur idPartie et idJoueur	Outils pour simuler différentes configurations
Disponibilité	Accessible uniquement si partie et joueur connus	Aucun
Stabilité	Résultats reproductibles sous base nettoyée	Scripts nettoyage

3. Évaluation synthétique

Critère	Note /5	Risques si non maîtrisé
Observabilité	5	Difficile de comprendre erreurs
Contrôlabilité	4	Limites si état partie non contrôlé
Disponibilité	5	Tests impossibles sans partie/joueur valide
Stabilité	5	Tests non fiables sans nettoyage

Fiche de testabilité —

requeteDonnerReponse(idPartie: Int, idJoueur: Int, cleJoueur: String, reponse: String)

Contexte du projet

Le joueur donne une réponse (oui/non) à une question posée.

1. Tableau de synthèse des critères

Critère	Évaluation	Justification synthétique
Observabilité	4/5	Retour EtatPartie, mais erreurs peu explicites
Contrôlabilité	4/5	Paramètres contrôlables, réponse limitée à "oui"/"non"
Disponibilité	4/5	Accessible en partie active et joueur authentifié
Stabilité	4/5	Stable si base nettoyée

2. Analyse détaillée par critère

Critère	Analyse détaillée	Pistes d'amélioration éventuelles
Observabilité	Résultat complet mais peu d'erreurs explicites	Messages d'erreur plus clairs
Contrôlabilité	Testable avec réponses valides (oui/non) et invalides	Validation stricte du paramètre "reponse"
Disponibilité	Nécessite partie en cours et joueur authentifié	Aucun
Stabilité	Résultats reproductibles avec nettoyage base	Nettoyage base avant tests

3. Évaluation synthétique

Critère	Note /5	Risques si non maîtrisé
Observabilité	4	Diagnostic limité
Contrôlabilité	4	Mauvaise gestion des réponses
Disponibilité	4	Échec si conditions non remplies
Stabilité	4	Tests non reproductibles

Fiche de testabilité — `playerLogin(lastName: String, name: String)`

Contexte du projet

Le projet comprend :

- un serveur fonctionnel,
- la bibliothèque cliente complète (`QuiEstCeClient`),
- un client incluant vue, contrôleur et modèle.

Ce contexte assure une bonne testabilité : les différents états peuvent être manipulés et le comportement du système observé.

1. Tableau de synthèse des critères

Critère	Évaluation (sur 5)	Justification synthétique
Observabilité	4	Le retour est exploitable ; les effets secondaires (création ou connexion) sont visibles
Contrôlabilité	4	Les entrées et les états du serveur et du fichier JSON sont maîtrisables
Disponibilité	5	Peut être testée dans tous les contextes
Stabilité	4	Résultats cohérents si les états initiaux sont bien réinitialisés

2. Analyse détaillée par critère

Critère	Analyse détaillée	Pistes d'amélioration éventuelles
Observabilité	La méthode retourne une paire de chaînes. On peut également vérifier les effets secondaires (création d'un joueur, mise à jour du fichier JSON).	Ajouter des messages de log explicites en cas de création ou d'erreur

Contrôlabilité	L'état local (via le fichier JSON) et distant (via le serveur) peut être préparé avant le test.	Intégrer un outil de réinitialisation automatique
Disponibilité	Cette méthode est autonome et ne dépend pas d'un état complexe. Elle peut être invoquée isolément.	Aucun
Stabilité	Les résultats sont reproductibles à condition de réinitialiser les fichiers et le serveur avant chaque test.	Prévoir un script de nettoyage avant test

3. Évaluation synthétique

Critère	Note sur 5	Risques si non maîtrisé
Observabilité	4	Difficile de savoir si un joueur a été créé ou simplement reconnu
Contrôlabilité	4	Risque de conflit si un joueur existe déjà côté serveur ou fichier local
Disponibilité	5	Aucun
Stabilité	4	Tests non reproductibles si l'environnement n'est pas correctement réinitialisé

Conclusion :

La méthode `playerLogin` est bien testable dans le projet actuel. Une maîtrise de l'environnement (état du fichier local, état du serveur) est nécessaire pour garantir des tests fiables et reproductibles.

Méthode : requeteCreationJoueur(nom: String, prenom: String)

Tableau1				
Champ	Classe d'équivalence	Valide / Invalide	Exemple d'entrée	
Nom	Lettres uniquement	Valide	« Dupont »	
Nom	Contient un apostrophe (caractère spécial valide)	Valide	« O'Connor »	
Nom	Contient un chiffre	Invalide	« Dupond1 »	
Nom	Contient un caractère spécial non autorisé	Invalide	« D@pont »	
Nom	Vide ou espaces seuls	Invalide	""	
Prénom	Lettres uniquement	Valide	« Jean »	
Prénom	Contient un apostrophe (caractère spécial valide)	Valide	« D'Arcy »	
Prénom	Contient un chiffre	Invalide	« J3an »	
Prénom	Contient un caractère spécial non autorisé	Invalide	« J#an »	
Prénom	Vide ou espaces seuls	Invalide	""	

Tableau2				
Tableau des cas de test	Colonne 1	Colonne 2	Colonne 3	Colonne 4
Cas	Nom	Prénom	Classes d'équivalence	Oracle attendu
1	« Durand »	« Jean »	Nom Valide (lettres) / Prénom Valide	Succès
2	« O'Connor »	« Jean »	Nom Valide (apostrophe) / Prénom Valide	Succès
3	« D4grand »	« Jean »	Nom Invalide (chiffre) / Prénom Valide	Échec
4	« Durand »	« D'Arcy »	Nom Valide / Prénom Valide (apostrophe)	Succès
5	« D@pont »	« Marie »	Nom Invalide (spécial non autorisé) / Prénom Valide	Échec
6	« Durand »	""	Nom Valide / Prénom Invalide (vide)	Échec
7	« D4grand »	« M@rie »	Nom Invalide (chiffre) / Prénom Invalide (spécial)	Échec

Méthode : requeteJoueur(idJoueur: Int)

Tableau3				
Tableau des classes d'équivalence	Colonne 1	Colonne 2	Colonne 3	
Champ	Classe d'équivalence	Valide / Invalide	Exemple d'entrée	
idJoueur	Entier positif (>= 0)	Valide		5
idJoueur	Entier négatif	Invalide		-1

Tableau4				
Tableau des cas de test	Colonne 1	Colonne 2	Colonne 3	
Cas	idJoueur	Classes d'équivalence	Oracle attendu	
1	10	Valide (positif)	Succès (joueur trouvé)	
2	0	Valide (0 est accepté)	Succès (joueur trouvé)	
3	-5	Invalide (négatif)	Échec (exception ou erreur)	

requetePoserQuestion(idPartie: Int, idJoueur: Int, cleJoueur: String, question: String)

Tableau5							
Tableau des classes	Colonne 1	Colonne 2	Colonne 3				
Champ	Classe d'équivalence	Valide / Invalide	Exemple d'entrée				
idPartie	Entier positif (>=0)	Valide		1			
idPartie	Entier négatif	Invalide		-1			
idJoueur	Entier positif (>=0)	Valide		3			
idJoueur	Entier négatif	Invalide		-10			
cleJoueur	Chaîne de 32 caractères	Valide	"12345678901234567890123456789012"				
cleJoueur	Chaîne < ou > 32 caractères	Invalide	"abc", ""				
question	Chaîne non vide, contenant uniquement des lettres, chiffres, espaces	Valide	"Le personnage a-t-il des lunettes ?"				
question	Chaîne vide	Invalide	""				
question	Contient caractères spéciaux non autorisés	Invalide	"Le personnage a-t-il des lunettes @ ?"				

Tableau6							
Tableau des cas de test	Colonne 1	Colonne 2	Colonne 3	Colonne 4	Colonne 5	Colonne 6	
Cas	idPartie	idJoueur	cleJoueur	question	Classes d'équivalence	Oracle attendu	
1	5	3	"12345678901234567890123456789012"	"Le personnage a-t-il les cheveux blonds ?"	Tous valides	Succès	
2	-1	3	"12345678901234567890123456789012"	"Le personnage porte-t-il un chapeau ?"	idPartie invalide	Échec	
3	5	-3	"12345678901234567890123456789012"	"Le personnage a-t-il des lunettes ?"	idJoueur invalide	Échec	
4	5	3	"12345"	"Le personnage a-t-il une barbe ?"	cleJoueur invalide	Échec	
5	5	3	"12345678901234567890123456789012"	""	question invalide (vide)	Échec	
6	5	3	"12345678901234567890123456789012"	"Le personnage a-t-il un menton pointu ?"	question valide avec caractères spéciaux valides	Succès	
7	5	3	"12345678901234567890123456789012"	"Le personnage a-t-il un tatouage @ ?"	question invalide avec caractère spécial non autorisé	Échec	

requeteDonnerReponse(idPartie: Int, idJoueur: Int, cleJoueur: String, reponse: String)

Tableau7							
Tableau des classe	Colonne 1	Colonne 2	Colonne 3				
Champ	Classe d'équivalence	Valide / Invalide	Exemple d'entrée				
idPartie	Entier positif (>= 0)	Valide	12				
idPartie	Entier négatif	Invalide	-3				
idJoueur	Entier positif (>= 0)	Valide	7				
idJoueur	Entier négatif	Invalide	-1				
cleJoueur	Chaîne de 32 caractères	Valide	"1234567890abcdef1234567890abcdef"				
cleJoueur	Chaîne < ou > 32 caractères	Invalide	"" , "abc"				
reponse	Valeur "oui" ou "non"	Valide	"oui", "non"				
reponse	Toute autre chaîne	Invalide	"maybe", "yes", "", "OUI"				
Tableau8							
Tableau des cas de	Colonne 1	Colonne 2	Colonne 3	Colonne 4	Colonne 5	Colonne 6	
Cas	idPartie	idJoueur	cleJoueur	reponse	Classes d'équivalence	Oracle attendu	
1	12	7	"1234567890abcdef1234567890abcdef"	"oui"	Tous valides	Succès : état partie mis à jour	
2	12	7	"1234567890abcdef1234567890abcdef"	"non"	Tous valides	Succès : état partie mis à jour	
3	-1	7	"1234567890abcdef1234567890abcdef"	"oui"	idPartie invalide	Exception ou échec	
4	12	-5	"1234567890abcdef1234567890abcdef"	"non"	idJoueur invalide	Exception ou échec	
5	12	7	"abc"	"oui"	cleJoueur invalide	Exception ou échec	
6	12	7	"1234567890abcdef1234567890abcdef"	"maybe"	reponse invalide	Exception ou échec	
7	12	7	"1234567890abcdef1234567890abcdef"	"OUI"	reponse invalide (casse différente)	Exception ou échec	

requeteChercherEncore(idPartie: Int, idJoueur: Int, cleJoueur: String): EtatPartie

Tableau9							
Tableau des classe	Colonne 1	Colonne 2	Colonne 3				
Champ	Classe d'équivalence	Valide / Invalide	Exemple d'entrée				
idPartie	Entier positif (>= 0)	Valide	3				
idPartie	Entier négatif	Invalide	-1				
idJoueur	Entier positif (>= 0)	Valide	5				
idJoueur	Entier négatif	Invalide	-10				
cleJoueur	Chaîne de 32 caractères	Valide	"abcdef1234567890abcdef1234567890"				
cleJoueur	Chaîne < ou > 32 caractères	Invalide	"" , "abc"				
Tableau10							
Tableau des cas de	Colonne 1	Colonne 2	Colonne 3	Colonne 4	Colonne 5		
Cas	idPartie	idJoueur	cleJoueur	Classes d'équivalence	Oracle attendu		
1		3	5	"abcdef1234567890abcdef1234567890"	Tous valides	Succès : retourne un EtatPartie valide	
2		-1	5	"abcdef1234567890abcdef1234567890"	idPartie invalide	Exception ou échec	
3		3	-10	"abcdef1234567890abcdef1234567890"	idJoueur invalide	Exception ou échec	
4		3	5	""	cleJoueur invalide	Exception ou échec	
5		3	5	"abc"	cleJoueur invalide	Exception ou échec	

requeteTrouve(idPartie: Int, idJoueur: Int, cleJoueur: String, ligne: Int, colonne: Int): Boolean

Tableau11							
Tableau des classe	Colonne 1	Colonne 2	Colonne 3				
Champ	Classe d'équivalence	Valide / Invalide	Exemple d'entrée				
idPartie	Entier positif (>= 0)	Valide	10				
idPartie	Entier négatif	Invalide	-1				
idJoueur	Entier positif (>= 0)	Valide	15				
idJoueur	Entier négatif	Invalide	-5				
cleJoueur	Chaîne de 32 caractères	Valide	"abcdef1234567890abcdef1234567890"				
cleJoueur	Chaîne < ou > 32 caractères	Invalide	"" , "abc"				
ligne	Entier positif dans la plage de la grille (0-3)	Valide	2				
ligne	Entier hors plage (négatif ou > 3)	Invalide	-1, 4				
colonne	Entier positif dans la plage de la grille (0-5)	Valide	3				
colonne	Entier hors plage (négatif ou > 5)	Invalide	-2, 6				
Tableau12							
Tableau des cas de	Colonne 1	Colonne 2	Colonne 3	Colonne 4	Colonne 5	Colonne 6	Colonne 7
Cas	idPartie	idJoueur	cleJoueur	ligne	colonne	Classes d'équivalence	Oracle attendu
1	10	15	"abcdef1234567890abcdef1234567890"	2	3	Tous valides	Retourne true ou false selon le jeu
2	-1	15	"abcdef1234567890abcdef1234567890"	2	3	idPartie invalide	Exception ou échec
3	10	-5	"abcdef1234567890abcdef1234567890"	2	3	idJoueur invalide	Exception ou échec
4	10	15	"abc"	2	3	cleJoueur invalide	Exception ou échec
5	10	15	"abcdef1234567890abcdef1234567890"	-1	3	ligne invalide	Exception ou échec
6	10	15	"abcdef1234567890abcdef1234567890"	2	6	colonne invalide	Exception ou échec

requeteEssai()

Tableau13			
Tableau des classes d'équi	Colonne 1	Colonne 2	Colonne 3
Champ	Classe d'équivalence	Valide / Invalide	Exemple d'entrée
Appel méthode	Serveur accessible, réponse reçue	Valide	N/A
Appel méthode	Serveur non accessible, timeout	Invalide	N/A
Appel méthode	Serveur accessible, réponse incorrecte	Invalide	N/A
Tableau14			
Tableau des cas de test	Colonne 1	Colonne 2	Colonne 3
Cas	Description	Classes d'équivalence	Oracle attendu
1	Appel normal, serveur OK	Serveur accessible, réponse reçue	Chaîne non vide, type attendu (ex: "pong")
2	Serveur non accessible	Serveur non accessible, timeout	Exception levée ou message d'erreur
3	Réponse incorrecte du serveur	Serveur accessible, réponse incorrecte	Message d'erreur ou gestion d'erreur

requeteJoueurs() : List<Int>

Tableau15			
Champ	Classe d'équivalence	Valide / Invalide	Exemple d'entrée
Réponse API	Liste non vide d'identifiants valides	Valide	[1, 2, 3, 10]
Réponse API	Liste vide	Valide	[]
Réponse API	Identifiants négatifs dans la liste	Invalide	[-1, 2, 3]
Réponse API	Identifiants non entiers (ex: chaînes, nulls)	Invalide	["a", null, 5]
Réponse API	Réponse du serveur non reçue / timeout	Invalide	N/A
Tableau16			
Tableau des cas de	Colonne 1	Colonne 2	Colonne 3
Cas	Description	Classes d'équivalence	Oracle attendu
1	Liste standard avec IDs positifs	Liste non vide d'identifiants valides	Liste reçue, IDs positifs uniquement
2	Liste vide (aucun joueur enregistré)	Liste vide	Liste vide reçue
3	Liste contenant un ID négatif	Identifiants négatifs dans la liste	Exception ou liste rejetée / filtrée
4	Liste contenant des valeurs invalides (type)	Identifiants non entiers	Exception ou erreur de parsing
5	Pas de réponse serveur (timeout, erreur)	Réponse du serveur non reçue	Exception levée ou gestion d'erreur

requeteCreationPartie(idJoueur: Int, cleJoueur: String): Int

Tableau17				
Tableau des classe	Colonne 1	Colonne 2	Colonne 3	
Champ	Classe d'équivalence	Valide / Invalide	Exemple d'entrée	
idJoueur	Entier positif	Valide		1
idJoueur	Zéro ou entier négatif	Invalide	-1, 0	
cleJoueur	Chaîne de 32 caractères	Valide	« abcd1234abcd1234abcd1234abcd1234 »	
cleJoueur	Moins de 32 caractères	Invalide	« abc123 »	
cleJoueur	Plus de 32 caractères	Invalide	« a...a » (40 caractères)	
cleJoueur	Chaîne vide ou espaces seuls	Invalide	« »	
cleJoueur	Contient des caractères spéciaux invalides	Invalide	« abc@123! »	
Tableau18				
Tableau des cas de	Colonne 1	Colonne 2	Colonne 3	Colonne 4
Cas	idJoueur	cleJoueur	Classes d'équivalence	Oracle attendu
1	5	« abcd1234abcd1234abcd1234abcd1234 »	idJoueur valide / cleJoueur valide	Succès, ID de partie reçu
2	-3	« abcd1234abcd1234abcd1234abcd1234 »	idJoueur invalide / cleJoueur valide	Échec
3	2	« abcd1234abcd »	idJoueur valide / cleJoueur trop court	Échec
4	1	« abcd1234abcd1234abcd1234abcd12345 »	idJoueur valide / cleJoueur trop long	Échec
5	3	« abc@123!@#abcd1234abcd1234abcd12 »	idJoueur valide / cleJoueur invalide (car. spéciaux)	Échec

requeteEtatPartie(int idPartie)

Tableau19			
Tableau des classe	Colonne 1	Colonne 2	Colonne 3
Champ	Classe d'équivalence	Valide / Invalide	Exemple d'entrée
idPartie	Entier positif (identifiant existant ou non)	Valide	1, 42
idPartie	Zéro ou entier négatif	Invalide	-1, 0

Tableau20			
Tableau des cas de	Colonne 1	Colonne 2	Colonne 3
Cas	idPartie	Classe d'équivalence	Oracle attendu
1	1	1 idPartie Valide	Succès - Retourne un objet EtatPartie
2	9999	idPartie Valide (mais inconnu)	Échec - Exception ou réponse d'erreur
3	-4	idPartie Invalide (négatif)	Échec - Exception IllegalArgumentExceptionException
4	0	idPartie Invalide (zéro)	Échec - Exception IllegalArgumentExceptionException

requeteRejoindrePartie(int idPartie, int idJoueur, String cleJoueur)

Tableau21				
Tableau des classe	Colonne 1	Colonne 2	Colonne 3	
Champ	Classe d'équivalence	Valide / Invalide	Exemple d'entrée	
idPartie	Entier ≥ 1	Valide	1, 42	
idPartie	Entier ≤ 0	Invalide	-1, 0	
idJoueur	Entier ≥ 1	Valide		3
idJoueur	Entier ≤ 0	Invalide	0, -2	
cleJoueur	Chaîne de 32 caractères	Valide	"12345678901234567890123456789012"	
cleJoueur	Longueur ≠ 32 caractères	Invalide	"court", "trop long....."	
cleJoueur	Chaîne vide ou contenant uniquement des espaces	Invalide	"", " "	

Tableau22					
Tableau des cas c	Colonne 1	Colonne 2	Colonne 3	Colonne 4	Colonne 5
Cas	idPartie	idJoueur	cleJoueur	Classes d'équivalence	Oracle attendu
1	1	1	3 "12345678901234567890123456789012"	idPartie Valide / idJoueur Valide / cleJoueur Valide	Succès - le joueur rejoint la partie
2		-1	3 "12345678901234567890123456789012"	idPartie Invalide / idJoueur Valide / cleJoueur Valide	Échec - exception
3		2	-5 "12345678901234567890123456789012"	idPartie Valide / idJoueur Invalide / cleJoueur Valide	Échec - exception
4		3	3 "abc"	idPartie Valide / idJoueur Valide / cleJoueur Invalide	Échec - exception
5		4	2 ""	idPartie Valide / idJoueur Valide / cleJoueur Invalide	Échec - exception
6		1	1 " 123"	idPartie Valide / idJoueur Valide / cleJoueur Invalide	Échec - exception

requeteChoixPersonnage(int idPartie, int idJoueur, String cleJoueur, int ligne, int colonne)

Tableau23

Tableau des class	Colonne 1	Colonne 2	Colonne 3
Champ	Classe d'équivalence	Valide / Invalide	Exemple d'entrée
idPartie	Entier strictement positif	Valide	1, 12
idPartie	Zéro ou entier négatif	Invalide	0, -5
idJoueur	Entier strictement positif	Valide	3
idJoueur	Zéro ou entier négatif	Invalide	-1, 0
cleJoueur	Chaîne de 32 caractères	Valide	"12345678901234567890123456789012"
cleJoueur	Longueur ≠ 32 ou chaîne vide	Invalide	"abc", "", ""
ligne	Valeur entre 0 et 3 (grille de 4 lignes)	Valide	0, 2
ligne	En dehors de [0-3]	Invalide	-1, 4
colonne	Valeur entre 0 et 5 (grille de 6 colonnes)	Valide	0, 5
colonne	En dehors de [0-5]	Invalide	-2, 6

Tableau24

Tableau des cas	Colonne 1	Colonne 2	Colonne 3	Colonne 4	Colonne 5	Colonne 6	Colonne 7
Cas	idPartie	idJoueur	cleJoueur	ligne	colonne	Classes d'équivalence	Oracle attendu
1		1	3 "12345678901234567890123456789012"	2	4	Tous valides	Succès - personnage choisi
2		-1	3 "12345678901234567890123456789012"	1	1	idPartie Invalide	Échec - exception
3		2	0 "12345678901234567890123456789012"	1	1	idJoueur Invalide	Échec - exception
4		2	2 "abc"	1	1	cleJoueur Invalide	Échec - exception
5		2	2 "12345678901234567890123456789012"	5	1	ligne Invalide	Échec - exception
6		2	2 "12345678901234567890123456789012"	1	6	colonne Invalide	Échec - exception
7		2	2 "12345678901234567890123456789012"	-1	-2	ligne Invalide / colonne Invalide	

requeteGrilleJoueur(int idPartie, int idJoueur)

Tableau25				
Tableau des classes d'équivalence	Colonne 1	Colonne 2	Colonne 3	
Champ	Classe d'équivalence	Valide / Invalide	Exemple d'entrée	
idPartie	Entier strictement positif	Valide	1, 7	
idPartie	Zéro ou entier négatif	Invalide	0, -3	
idJoueur	Entier strictement positif	Valide	2, 10	
idJoueur	Zéro ou entier négatif	Invalide	-1, 0	
Tableau26				
Tableau des cas de test	Colonne 1	Colonne 2	Colonne 3	Colonne 4
Cas	idPartie	idJoueur	Classes d'équivalence	Oracle attendu
	1	1	2 idPartie Valide / idJoueur Valide	Succès - retourne la grille
	2	-1	2 idPartie Invalide / idJoueur Valide	Échec - exception
	3	3	-5 idPartie Valide / idJoueur Invalide	Échec - exception
	4	0	0 idPartie Invalide / idJoueur Invalide	Échec - exception

requeteListeParties()

Tableau27				
Tableau des classes d'équivalence	Colonne 1	Colonne 2	Colonne 3	
Champ (état système)	Classe d'équivalence	Valide / Invalide	Exemple d'entrée	
Parties existantes	Aucune partie présente	Valide	0 partie dans la base	
Parties existantes	Une ou plusieurs parties existantes	Valide	1, 2, ... identifiants disponibles	
Parties existantes	Base corrompue ou inaccessible (hypothèse rare)	Invalide	Erreur de connexion à la base	
Tableau28				
Tableau des cas de test	Colonne 1	Colonne 2	Colonne 3	
Cas	État du système	Classes d'équivalence	Oracle attendu	
	1 Aucune partie enregistrée	Parties existantes Valide (0 partie)	Retourne une liste vide	
	2 Deux parties créées	Parties existantes Valide	Retourne une liste contenant 2 identifiants	
	3 Serveur non disponible ou erreur base	Parties existantes Invalide	Échec - exception levée (ex : IOException)	

requeteListePartiesCreees()

Tableau29				
Tableau des classes d'équivalence	Colonne 1	Colonne 2	Colonne 3	
Champ	Classe d'équivalence	Valide / Invalide	Exemple d'entrée	
Aucun paramètre	Méthode sans paramètre	Valide	—	
Retour	Liste vide	Valide	[]	
Retour	Liste non vide avec identifiants valides	Valide	[1, 5, 12]	
Retour	Liste contenant identifiants négatifs (impossible non	Invalide (logique)	[-1, 0, 3]	
Retour	Liste contenant doublons	Invalide (selon logique métier)	[3, 3, 7]	
Tableau30				
Tableau des cas de test	Colonne 1	Colonne 2	Colonne 3	
Cas	Entrée	Classes d'équivalence	Oracle attendu	
	1 (aucune)	Méthode sans paramètre / Liste vide	Liste vide, succès	
	2 (aucune)	Méthode sans paramètre / Liste non vide avec id valides	Liste non vide, succès	
	3 (aucune)	Méthode sans paramètre / Liste avec id négatifs (hypothétique)	Échec ou correction des id	
	4 (aucune)	Méthode sans paramètre / Liste avec doublons (hypothétique)	Échec ou filtrage des doublons	

requeteListePartiesTerminees()

Tableau31				
Tableau des classes d'équivalence	Colonne 1	Colonne 2	Colonne 3	
Champ	Classe d'équivalence	Valide / Invalide	Exemple d'entrée	
Aucun paramètre	Méthode sans paramètre	Valide	—	
Retour	Liste vide	Valide	[]	
Retour	Liste non vide avec identifiants valides	Valide	[2, 4, 9]	
Retour	Liste contenant identifiants négatifs (impossible normalement)	Invalide (logique)	[-2, 0, 3]	
Retour	Liste contenant doublons	Invalide (selon logique métier)	[5, 5, 7]	
Tableau32				
Tableau des cas de test	Colonne 1	Colonne 2	Colonne 3	
Cas	Entrée	Classes d'équivalence	Oracle attendu	
	1 (aucune)	Méthode sans paramètre / Liste vide	Liste vide, succès	
	2 (aucune)	Méthode sans paramètre / Liste non vide avec id valides	Liste non vide, succès	
	3 (aucune)	Méthode sans paramètre / Liste avec id négatifs (hypothétique)	Échec ou correction des id	
	4 (aucune)	Méthode sans paramètre / Liste avec doublons (hypothétique)	Échec ou filtrage des doublons	

QuiEstCeClient(hote: String, port: Int)

Tableau33				
Tableau des classes d'éq	Colonne 1	Colonne 2	Colonne 3	
Champ	Classe d'équivalence	Valide / Invalide	Exemple d'entrée	
hote	Nom de domaine ou adresse IP valide	Valide	"localhost", "192.168.1.1", "example.com"	
hote	Chaîne vide ou espaces	Invalide	"", " "	
hote	Nom de domaine avec caractères spéciaux valides (apostrophe, tiret)	Valide	"mon-site.com", "o'connor.net"	
hote	Nom de domaine avec caractères invalides	Invalide	"site@123", "ex!ample"	
port	Entier dans l'intervalle valide (1–65535)	Valide	80, 8080, 443	
port	Entier hors de l'intervalle valide (ex. négatif, 0, >65535)	Invalide	-1, 0, 70000	
Tableau34				
Tableau des cas de test	Colonne 1	Colonne 2	Colonne 3	Colonne 4
Cas	hote	port	Classes d'équivalence	Oracle attendu
1 "localhost"		8080	hote valide / port valide	Succès : instance créée
2 ""		8080	hote invalide (vide) / port valide	Échec : exception ou erreur
3 "example.com"		70000	hote valide / port invalide (hors plage)	Échec : exception ou erreur
4 "mon-site.com"		443	hote valide (avec tiret) / port valide	Succès : instance créée
5 "site@123"		80	hote invalide (caractères invalides) / port valide	Échec : exception ou erreur
6 "o'connor.net"		0	hote valide (avec apostrophe) / port invalide	Échec : exception ou erreur

METHODE DU MODELE CLIENT

fun playerLogin(lastName: String, name: String): Pair<String, String>

Tableau35				
fun playerLogin(lastName:	Colonne 1	Colonne 2	Colonne 3	
Tableau des classes d'équivalence				
Champ	Classe d'équivalence	Valide / Invalide	Exemple d'entrée	
lastName	Lettres uniquement	Valide	"DUPONT"	
lastName	Contient un chiffre	Invalide	"DUP0NT"	
lastName	Contient un caractère spécial invalide	Invalide	"DUP@NT"	
lastName	Contient un caractère spécial valide (ex: tiret)	Valide	"DUPONT-MARTIN"	
lastName	Vide ou espaces seuls	Invalide	""	
name	Lettres uniquement	Valide	"Jean"	
name	Contient un chiffre	Invalide	"J3an"	
name	Contient un caractère spécial invalide	Invalide	"J@an"	
name	Contient un caractère spécial valide	Valide	"Jean-paul"	
name	Vide ou espaces seuls	Invalide	""	
Tableau36				
Tableau des cas de test	Colonne 1	Colonne 2	Colonne 3	Colonne 4
Cas	lastName	name	Classes d'équivalence	Oracle attendu
1 "DUPONT"		"Jean"	lastName Valide / name Valide	Connexion ou création
2 "DUPONT"		"Jean"	lastName Invalide (chiffre) / name Valide	Exception ou rejet
3 "DUPONT"		"J3an"	lastName Valide / name Invalide (chiffre)	Exception ou rejet
4 "DUP@NT"		"Jean"	lastName Invalide (caractère spécial) / name Valide	Exception ou rejet
5 "DUPONT"		"Jean-paul"	lastName Valide / name Valide (caractère spécial valide)	Connexion ou création

fun matchCreate(): Match

Tableau37

Tableau des classes d'équivalence

Champ	Classe d'équivalence	Valide / Invalide	Exemple d'entrée	
currentPlayer.id	Identifiant strictement positif	Valide		5
currentPlayer.id	Négatif ou nul	Invalide		0
currentPlayer.cle	Chaîne non vide	Valide	"cle123"	
currentPlayer.cle	Chaîne vide	Invalide	""	
currentPlayer	Joueur connecté (initialisé via playerLogin)	Valide	Joueur("Durand", "Jean")	
currentPlayer	Joueur non initialisé (pas passé par playerLogin)	Invalide	Joueur("", "")	

Tableau38

Tableau des cas de test

Cas	currentPlayer.id	currentPlayer.cle	currentPlayer initialisé	Classes d'équivalence	Oracle attendu
1	5	"cle123"	Oui	id Valide / cle Valide / joueur initialisé	Création du match OK
2	0	"cle123"	Oui	id Invalide / cle Valide	Exception ou erreur serveur
3	5	""	Oui	id Valide / cle Invalide	Exception ou erreur serveur
4	5	"cle123"	Non	id Valide / cle Valide / joueur non initiali	Exception QuiEstCeException